

VAID 项目架构与安全审查说明

生成日期：2026-05-02

用途：发送给 Google Gemini，请其作为外部架构与安全顾问复核当前判断和修复方案。

1. 执行摘要

VAID / V-ID 是一个为数字资产、虚拟角色、原创作品生成“数字身份证书 / 存证证书”的网站。用户上传图片并填写角色名、创作者名后，系统生成证书、写入 Supabase 数据库，并生成可公开验证的二维码。项目还接入支付宝支付，用户付费后获得激活码或额度，用于生成更多证书。系统还包含 OTS 存证相关功能。

本次代码审查发现：项目目前不是“跑不起来”的问题，而是存在较明显的业务安全与架构风险。
`npm run lint`、`npm run typecheck`、生产构建均已通过，但后端授权边界不够严格，部分关键业务动作仍由前端直接发起或由前端传参决定。

最核心的问题是：前端权力过大，数据库和 Edge Functions 过度信任前端传来的 `client_id`、`friendly_id`、`sha256_hash`、`out_trade_no` 等字段。这样会导致绕过支付、刷免费额度、伪造 OTS 存证、查询他人订单激活码等风险。

我的初步结论是：如果只是临时上线，可以做一版加固；但如果要彻底解决旧架构遗留问题，必须引入账号系统，并把“扣额度、写数据库、查订单、发激活码、触发 OTS”的关键动作收口到后端。

2. 项目技术背景

项目路径：

`/Users/yan/Documents/VAID/Vid-full-main`

主要技术栈：

- 前端：Vite + React + TypeScript
- 部署：Vercel
- 后端：Supabase
- 数据库：Supabase Postgres
- 鉴权：目前没有正式账号登录系统
- 支付：支付宝
- 后端函数：Supabase Edge Functions

- 存证：OTS 相关函数
- 图片存储：Supabase Storage

主要代码文件：

```
src/App.tsx
src/components/CardGenerator.tsx
src/pages/VerifyPage.tsx
src/pages/PaymentSuccessPage.tsx
src/utils/licenseManager.ts
src/utils/supabase.ts
supabase/functions/alipay-create-order/index.ts
supabase/functions/alipay-notify/index.ts
supabase/functions/alipay-query-order/index.ts
supabase/functions/quota-check/index.ts
supabase/functions/quota-use/index.ts
supabase/functions/license-key-status/index.ts
supabase/functions/license-key-use/index.ts
supabase/functions/ots-stamp/index.ts
supabase/functions/ots-verify/index.ts
supabase/migrations/*.sql
```

3. 项目功能背景

网站的核心产品逻辑大致如下：

1. 用户访问网站。
2. 用户上传角色图片。
3. 用户填写角色名、创作者名。
4. 系统计算或保存文件 hash。
5. 系统生成一个证书视觉图。
6. 系统生成一个 `friendly_id`。
7. 系统把证书记录写入 `v_ids` 数据表。
8. 系统生成二维码，二维码指向 `/verify/{friendly_id}`。
9. 验证页通过 `friendly_id` 查询数据库并展示记录。
10. 用户可购买套餐或使用激活码获得更多生成次数。
11. 系统通过 OTS 为 hash 生成存证文件。

历史上，项目先追求 MVP 跑通：生成证书、支付、验证页、免费额度、激活码、OTS 存证等功能逐步叠加。后面也经历过支付成功页卡住、额度扣减失败等问题，因此做过一些稳定性兜底改动。

这些改动让项目流程可用，但留下了架构债：很多安全关键动作仍然依赖前端传参，后端没有完整验证调用者身份和资源归属。

4. 当前架构现状

4.1 免费额度

当前免费额度大致绑定到客户端生成的 `client_id` 或浏览器指纹。前端调用：

```
quota-check  
quota-use
```

后端根据前端提交的 `client_id` 检查或创建额度记录。如果没有该 `client_id`，就创建一条新记录并给 3 次免费额度。

问题是：`client_id` 来自客户端，攻击者可以伪造或反复更换。

4.2 支付和激活码

用户选择套餐后，前端调用：

```
alipay-create-order
```

支付完成后进入：

```
/payment-success
```

支付成功页调用：

```
alipay-query-order
```

查询订单支付状态并显示激活码。

激活码相关函数：

```
license-key-status  
license-key-use
```

当前风险点是：订单查询主要依赖 `out_trade_no`，没有严格绑定登录用户，因为系统目前没有正式账号体系。

4.3 证书生成

当前 `CardGenerator.tsx` 中仍存在前端直接写入 Supabase 表的逻辑：

```
supabase  
  .from('v_ids')  
  .insert({  
    character_name,  
    creator_name,
```

```
sha256_hash,  
original_file_hash,  
image_url,  
friendly_id,  
ots_status: 'pending',  
})
```

这意味着前端拥有写入证书主表的能力。

4.4 OTS 存证

`ots-stamp` 函数接受前端传入的：

```
friendly_id  
sha256_hash
```

然后用 Supabase service role：

1. 生成 OTS 文件。
2. 上传到 Storage。
3. 更新 `v_ids.ots_status` 和 `v_ids.ots_file_path`。

当前风险点是：后端没有强制从数据库读取真实 hash，而是信任前端传来的 `sha256_hash`。

5. 历史来龙去脉

项目历史文档显示，这些问题不是最近 UI 调整造成的，也不是某一次小改误伤。它们更像是“先把流程跑通”后留下的旧架构遗留问题。

5.1 早期目标是 MVP 跑通

早期重点是让用户能上传图片、生成证书、写入数据库、通过二维码验证。前端直接写数据库是最快的实现方式。

5.2 支付事故修复优先

项目曾出现支付成功页卡住的问题。原因是代码和生产数据库 schema 不一致：代码查询 `alipay_orders.license_key`，但生产数据库当时没有该字段。

后续为了兼容旧 schema 和新 schema，新增或强化了 `alipay-query-order`，用于主动查询支付宝订单状态并同步本地订单。

这个修复解决了“支付成功页卡住”，但没有彻底补上“订单只能由创建它的用户查询”的权限边界。

5.3 生成流程稳定性优先

项目曾出现用户点击下一步没有明显反应的问题。原因之一是 `quota-use` 可能失败。后来为了稳定生成流程，`quota-use` 增加了 `fallback`：如果 RPC 扣额度失败，则直接读写 `user_quotas` 表扣额度。

这个改动提高了稳定性，但仍然信任前端传来的 `client_id`，没有解决 `client_id` 可伪造的问题。

5.4 一段时间明确不改后端架构

历史记录中有阶段性约束：只改视觉和 UI，不动支付、数据库、Supabase 架构。这使得前端直写 `v_ids`、客户端 `client_id`、公开 OTS 函数等旧设计一直保留下来。

6. 已完成的验证

本地检查结果：

```
npm run lint: 通过
npm run typecheck -- --pretty false: 通过
npm run build -- --outDir /private/tmp/vaid-build-check --emptyOutDir: 通过
```

构建有一个性能警告：主 JS chunk 约 542 kB，超过 Vite 默认 500 kB 警告线。这属于性能优化项，不是阻塞 bug。

依赖审计：

```
npm audit --audit-level=moderate
```

结果发现 16 个漏洞，其中：

```
6 high
9 moderate
1 low
```

涉及依赖包括：

```
rollup
glob
minimatch
picomatch
cross-spawn
flatted
esbuild
vite
postcss
js-yaml
yaml
nanoid
```

多数属于构建链依赖，但仍应升级并回归测试。

7. 主要安全和业务问题

7.1 P1：v_ids 可以被匿名写入，绕过额度和支付

相关位置：

supabase/migrations/20260324153351_fix_security_and_performance_issues.sql

当前 RLS policy 允许匿名用户插入 v_ids，只要字段满足基本校验：

- character_name 非空
- creator_name 非空
- sha256_hash 非空
- sha256_hash 长度为 64

由于 Supabase anon key 必须暴露给前端，任何访问网站的人都能拿到 anon key。如果 RLS 允许 anon role 插入，则攻击者可以绕过网站 UI，直接通过 Supabase REST API 插入证书记录。

风险：

- 绕过免费额度。
- 绕过付费流程。
- 绕过激活码。
- 批量灌入垃圾证书。
- 抢占某些 hash。
- 污染公开验证页数据。

根本原因：证书注册这个关键业务动作放在前端直接做，数据库也允许匿名插入。

建议修复：

- 关闭 v_ids 匿名 INSERT。
- 新建 v-id-register Edge Function。
- 前端生成证书时只调用 v-id-register。
- 后端校验用户身份、额度、激活码、hash、重复记录后，再用 service role 写入数据库。

7.2 P1：OTS 存证接口可以被伪造

相关位置：

supabase/functions/ots-stamp/index.ts

当前 **ots-stamp** 接收：

```
const { friendly_id, sha256_hash } = await req.json();
```

然后直接基于传入的 **sha256_hash** 生成 OTS，并更新对应 **friendly_id** 的存证状态。

风险：

- 攻击者知道某个 **friendly_id** 后，可以传入任意 hash。
- 后端可能为错误 hash 生成 OTS 文件。
- **v_ids** 中的 OTS 状态可能被错误更新。
- 证书存证可信度被破坏。

根本原因：**ots-stamp** 信任前端传入的 hash，而没有从数据库读取真实 hash。

建议修复：

- **ots-stamp** 不再接受前端传入的 **sha256_hash**。
- 只接受 **friendly_id** 或内部任务 ID。
- 后端根据 **friendly_id** 查询 **v_ids**。
- 从数据库记录中读取真实 **sha256_hash**。
- 使用数据库中的 hash 生成 OTS。
- 最好只由 **v-id-register** 后端流程内部触发，不直接暴露给浏览器。

7.3 P1：支付订单查询可能泄露激活码

相关位置：

supabase/functions/alipay-query-order/index.ts
src/pages/PaymentSuccessPage.tsx

当前支付成功页调用 **alipay-query-order** 时主要提交 **out_trade_no**。后端按订单号查询订单，成功后可能返回包含 **license_key** 的订单对象。

风险：

- 如果别人知道订单号，就可能查到激活码。
- 订单号可能出现在 URL、日志、截图、浏览器历史、Referrer 中。

- 激活码属于敏感权益，不应只凭订单号查询。

根本原因：订单查询没有绑定可信用户身份。

建议彻底修复：

- 引入登录系统。
- 创建订单时记录 `user_id`。
- 查询订单时从 JWT 获取当前 `user_id`。
- 只有订单所属用户才能查询订单敏感信息。
- 不匹配时不返回 `license_key`。

过渡加固：

- 创建订单时记录 `client_id`。
- 查询订单时必须提交同一个 `client_id`。
- 后端校验 `client_id` 与订单记录一致。
- 这不是彻底方案，因为 `client_id` 仍来自客户端。

7.4 P1：免费额度可以被刷

相关位置：

```
supabase/functions/quota-check/index.ts  
supabase/functions/quota-use/index.ts
```

当前逻辑信任前端提交的 `client_id`。如果数据库没有这个 `client_id`，就创建一条额度记录并给 3 次免费额度。

攻击者可以反复更换 `client_id`，从而不断获得免费额度。

风险：

- 免费额度无法真正限制。
- 可能被批量生成证书。
- 可能增加 Storage、OTS、数据库成本。
- 可能污染公开证书记录。

根本原因：额度绑定的是客户端自报身份，而不是可信身份。

建议彻底修复：

- 引入 Supabase Auth。
- 额度绑定 `auth.users.id`。
- 免费额度、付费额度、激活码兑换全部进入用户账户。

过渡加固：

- 保留 `client_id`，但增加 IP 限流、设备指纹、失败次数限制、频率限制。
- 这只能提高滥用成本，不能彻底防刷。

8. 其他问题

8.1 P2：数据库写入失败时，前端仍可能显示证书生成成功

相关位置：

`src/components/CardGenerator.tsx`

当前逻辑中，如果 `v_ids` 插入失败，除了重复 hash 的特殊处理之外，前端仍可能设置 `citizenId` 和二维码 URL，导致用户看到一个看似成功的证书。

风险：

- 用户以为证书生成成功。
- 验证页查不到该记录。
- 额度可能已经被消耗。
- 业务状态不一致。

建议：

- 非重复错误应立即阻断生成。
- 给用户明确错误提示。
- 如果额度已扣，应补偿或回滚。
- 最好把扣额度和写证书放到同一个后端事务或同一个可信 RPC 中。

8.2 P2：运行时从 CDN 动态加载 JSZip

相关位置：

`src/components/CardGenerator.tsx`
`src/pages/VerifyPage.tsx`

当前存在：

```
await import('https://cdn.jsdelivr.net/npm/jszip@3.10.1/+esm')
```

风险：

- CDN 不可用时下载功能失败。
- CDN 被污染时存在供应链风险。
- 当前 `vercel.json` 没有严格 CSP。

建议：

- 将 `jszip` 加入 `package.json`。
- 使用本地 `import`。
- 构建时打包。
- 增加 Content-Security-Policy。

8.3 P2：依赖审计存在 high 漏洞

`npm audit` 发现 16 个漏洞，其中 6 个 high。建议：

1. 执行 `npm audit fix`。
2. 如有破坏性升级，逐个升级依赖。
3. 复跑 `lint`、`typecheck`、`build`。
4. 手动验证核心流程。

8.4 P3：上传超大图片可能卡死浏览器

相关位置：

```
src/App.tsx  
src/utils/imageUpload.ts
```

当前上传时先 `FileReader.readAsDataURL(file)`，后续才压缩。超大文件会先进入内存，移动端可能卡死。

建议：

- 在读取前检查 `file.size`。
- 限制 MIME 类型。
- 限制最大图片尺寸或最大字节数。

- 给用户明确提示。

8.5 代码库卫生问题

supabase/.temp/* 文件被 git 跟踪，其中包括 Supabase 项目元数据，例如 pooler URL、project ref 等。建议加入 .gitignore 并从 Git 索引中移除。

9. 根本问题判断

这些问题的共同根因不是某一段代码写错，而是架构边界不清晰：

前端承担了太多关键业务权力。
数据库对匿名用户开放了不该开放的写权限。
Edge Functions 过度信任前端传来的身份和业务字段。
没有正式用户身份体系，导致额度、订单、证书归属无法可靠绑定。

因此，如果只修某一处，例如只给 alipay-query-order 加 client_id 校验，只能算加固，不能算彻底解决。

彻底方案应该是：

引入账号系统。
关键业务动作全部迁移到后端。
数据库只信任后端和经过 RLS 限制的登录用户。
前端只负责展示和提交请求。

10. 推荐的彻底解决方案

10.1 引入 Supabase Auth

用户需要登录后才能：

- 生成证书
- 使用免费额度
- 购买额度
- 兑换激活码
- 查询自己的订单
- 管理自己的证书

建议登录方式：

邮箱验证码 / Magic Link
或 邮箱 + 密码

暂时不建议短信登录，因为短信登录会带来额外成本和风控问题。

10.2 新建 profiles 表

示例：

```
create table profiles (  
  id uuid primary key references auth.users(id) on delete cascade,  
  email text,  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

10.3 新建或重构 user_credits 表

示例：

```
create table user_credits (  
  user_id uuid primary key references auth.users(id) on delete cascade,  
  free_credits integer not null default 3,  
  paid_credits integer not null default 0,  
  total_used integer not null default 0,  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

额度绑定 `user_id`，不再绑定前端自报的 `client_id`。

10.4 v_ids 表绑定 user_id

`v_ids` 增加：

```
user_id uuid references auth.users(id)
```

RLS 原则：

- 公开验证页可以读取必要公开字段。
- 登录用户可以读取自己的完整证书。
- 普通用户不能直接 insert。
- 普通用户不能 update 关键字段。
- 后端 service role 才能 insert/update 关键字段。

10.5 新建统一注册函数 v-id-register

这是最关键的重构。

前端旧逻辑：

```
supabase.from('v_ids').insert(...)
```

前端新逻辑：

```
supabase.functions.invoke('v-id-register', { body: payload })
```

v-id-register 负责：

1. 从 Authorization header 读取 JWT。
2. 调用 Supabase Auth 获取当前用户。
3. 校验用户已登录。
4. 校验输入字段。
5. 校验图片路径是否属于该用户。
6. 校验或计算 hash。
7. 检查重复 hash。
8. 检查用户额度。
9. 在后端扣额度。
10. 写入 **v_ids**。
11. 生成或确认 **friendly_id**。
12. 创建 OTS 任务或触发 OTS。
13. 返回最终证书 ID。

要求：扣额度和写证书必须尽量原子化。理想做法是封装成 Postgres RPC，在一个数据库事务里完成。

10.6 重写 OTS 流程

新流程：

1. **v-id-register** 成功写入 **v_ids**。
2. 后端创建 OTS 任务或直接调用内部 OTS 流程。
3. **ots-stamp** 不接受前端 hash。
4. **ots-stamp** 根据 **friendly_id** 查询数据库。
5. 从数据库读取真实 **sha256_hash**。
6. 生成 OTS 文件。
7. 上传 Storage。
8. 更新 **ots_status** 和 **ots_file_path**。

建议将 OTS 改为异步任务，避免用户等待太久。

10.7 支付订单绑定 user_id

alipay_orders 增加：

```
user_id uuid references auth.users(id)
```

创建订单：

1. 用户必须登录。
2. 后端从 JWT 获取 user_id。
3. 订单记录绑定 user_id。
4. 不再依赖前端 client_id 作为核心身份。

查询订单：

1. 用户必须登录。
2. 后端从 JWT 获取 user_id。
3. 只允许查询属于自己的订单。
4. 不匹配时不返回订单敏感信息和激活码。

10.8 激活码改成兑换到账户

建议保留激活码，但改变它的使用方式。

旧模式：生成时直接消耗激活码。

新模式：

1. 用户购买后获得激活码。
2. 用户登录账户。
3. 输入激活码。
4. 后端验证激活码。
5. 激活码兑换为账户 paid_credits。
6. 激活码标记为已兑换，并绑定 redeemed_by_user_id。
7. 后续生成证书只消耗账户额度。

好处：

- 用户换设备也能使用。
- 权益和账户绑定。
- 更容易审计。
- 更容易防止激活码被重复使用。

10.9 图片上传受控

建议：

1. 前端读取前限制文件大小。
2. 前端限制 MIME 类型。
3. Storage 路径带上 `user_id`：

```
avatars/{user_id}/{uuid}.jpg
```

4. Storage policy 限制用户只能上传到自己的目录。
5. 后端写证书时确认图片 URL 属于当前用户。
6. 防止用户引用别人上传的图片路径。

10.10 增加限流和审计日志

关键接口应记录：

```
user_id  
ip  
user_agent  
action  
success/failure  
error_code  
created_at
```

关键接口包括：

```
v-id-register  
alipay-create-order  
alipay-query-order  
license-key-use  
ots-stamp  
quota-use / credit-use
```

应增加限制：

- 同一用户每分钟生成次数限制。
- 同一 IP 每分钟请求限制。
- 同一激活码错误尝试次数限制。

- 同一订单查询频率限制。

11. 推荐迁移路线

阶段 1：准备 Auth 和新表

1. 开启 Supabase Auth。
2. 前端加入登录/退出 UI。
3. 新建 `profiles`。
4. 新建 `user_credits`。
5. `v_ids` 增加 `user_id`。
6. `alipay_orders` 增加 `user_id`。
7. 设计新的 RLS policy。

阶段 2：新增安全后端函数

新增：

```
v-id-register
```

先在测试环境完成：

- 校验登录
- 检查额度
- 扣额度
- 写 `v_ids`
- 触发 OTS

阶段 3：前端切换到新函数

修改 `CardGenerator.tsx`，不再直接执行：

```
supabase.from('v_ids').insert(...)
```

改为调用：

```
supabase.functions.invoke('v-id-register', ...)
```

阶段 4：关闭旧数据库权限

关闭：

v_ids 匿名 INSERT
v_ids 匿名 UPDATE

只保留公开验证页所需的 SELECT 权限。

阶段 5：支付流程绑定 user_id

修改：

alipay-create-order
alipay-query-order
alipay-notify
PaymentSuccessPage
licenseManager

目标：

- 创建订单时绑定用户。
- 查询订单时验证用户。
- 激活码只返回给订单所属用户。
- 支付成功后权益进入账户。

阶段 6：重写 OTS

修改：

ots-stamp

目标：

- 不接收前端 hash。
- 从数据库读取 hash。
- 只由后端可信流程触发。
- 防止任意人覆盖 OTS 状态。

阶段 7：清理旧逻辑

清理：

- 基于 client_id 的核心免费额度逻辑。
- 旧 quota-check。

- 旧 quota-use。
- 前端直写 v_ids。
- 可公开调用的危险 OTS 逻辑。
- 匿名 insert policy。
- 旧测试数据。
- supabase/.temp/* 这类不应被 Git 跟踪的文件。

阶段 8：补测试

至少测试：

1. 未登录用户不能生成证书。
2. 登录用户首次有免费额度。
3. 免费额度用完后不能继续免费生成。
4. 购买后订单绑定当前用户。
5. A 用户不能查询 B 用户订单。
6. A 用户不能拿 B 用户激活码。
7. 激活码只能兑换一次。
8. 证书写入失败时不扣额度或可补偿。
9. OTS 只使用数据库中的 hash。
10. 攻击者不能直接插入 v_ids。
11. 攻击者不能直接更新 v_ids。
12. 验证页仍可公开查看必要信息。
13. 上传超大图片会被前端拒绝。
14. CDN 断网不影响 ZIP 下载功能。

12. 成本影响判断

增加登录系统会带来一些成本，但不一定很高。

如果使用 Supabase Auth：

- Free 计划有一定 MAU 免费额度。
- Pro / Team 有更高 MAU 包含额度。
- 超出后按月活用户计费。
- 如果不用短信登录，成本通常可控。

真正可能增加成本的是：

1. 用户量增长后的 Auth MAU。
2. Edge Function 调用次数增加。
3. Storage 图片存储增长。
4. OTS 存证调用增长。
5. 如果使用短信验证码，会产生短信成本。

建议：

先用邮箱登录，不用短信。
先把安全架构修好，再考虑更复杂的登录方式。

13. 给 Gemini 的具体问题

请你作为资深全栈 / Supabase / 安全架构顾问，帮忙评估：

1. 当前判断是否正确：最大问题是否是前端权力太大，应该把证书注册、扣额度、支付订单查询、OTS 存证全部收口到后端？
2. 如果使用 Supabase Auth，这套重构方案是否合理？
3. `v-id-register` 应该如何设计，才能保证扣额度和写入 `v_ids` 的一致性？
4. Supabase Edge Function 中如何安全获取当前登录用户？
5. `v_ids` 的 RLS policy 应该如何设计，才能同时满足公开验证页读取和防止匿名写入？
6. 支付订单和激活码应该绑定 `user_id`，还是有更好的设计？
7. 是否应该保留激活码？还是支付后直接给账户加额度更好？
8. OTS 存证应该同步执行，还是写入队列表异步执行？
9. 如果暂时不做账号系统，有没有足够安全的过渡方案？
10. 这个项目如果要上线真实支付，最低必须先修哪些问题？
11. Supabase 上实现“扣额度 + 写证书”的原子事务，推荐用 Postgres RPC 还是 Edge Function 内多步操作？
12. 对公开验证页，哪些字段应该公开，哪些字段应该只允许证书所有者查看？
13. 对激活码、订单查询、OTS 存证，是否需要额外引入 Cloudflare / Vercel 层限流？
14. 当前依赖漏洞主要在构建链，是否需要作为上线阻塞项处理？

14. 我的初步结论

如果只是短期加固，可以做：

- client_id 校验
- IP 限流
- 关闭危险公开接口
- OTS 从数据库读取 hash
- 订单查询校验 client_id
- 前端不再在 DB 失败后显示成功

但这只能降低风险，不能彻底解决。

如果目标是彻底解决旧架构遗留问题，应做：

- 引入 Supabase Auth
- 额度绑定 user_id
- 订单绑定 user_id
- 证书绑定 user_id
- 前端不再直写 v_ids
- 新增 v-id-register 后端函数
- OTS 改为后端可信流程
- 激活码兑换到账户
- 重写 RLS policy
- 补限流和审计日志

最终判断：

当前项目的 UI 和基础流程可以继续完善，但如果要上线真实支付和真实存证，必须先收紧后端权限边界。最根本的修复不是局部补丁，而是把核心业务动作从前端迁移到后端，并引入可信用户身份。